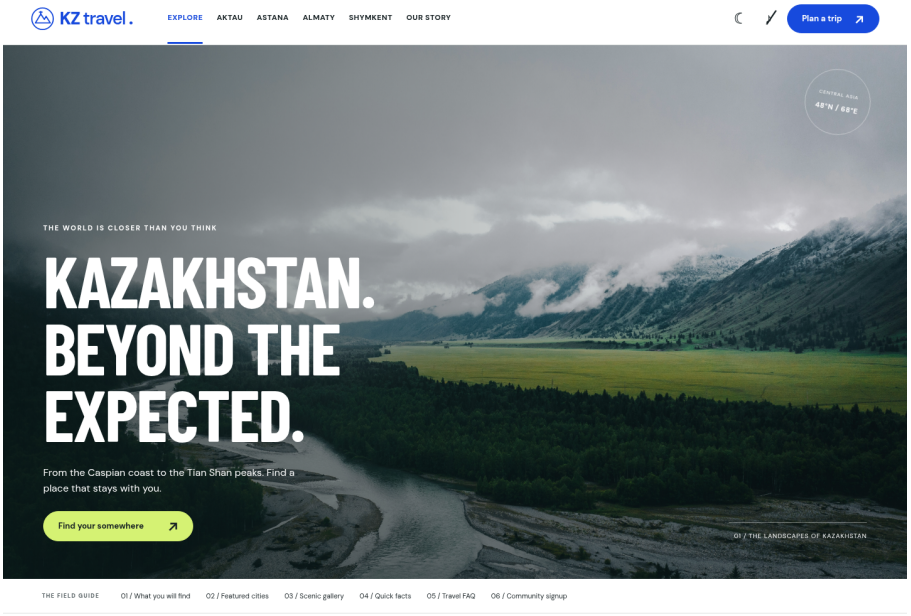
Go a little further. Assignment 5

Advanced JavaScript and DOM Manipulation



KZ Travel: the published Kazakhstan guide using the original project photographs.

Objective. Continue the six-page travel project with an engaging interface that demonstrates JavaScript fundamentals, DOM manipulation, events, callbacks, data structures, sound and animations.

Team	KZ Travel
Members	Ramazan Utegen Ziyadinkhan Kudaibergenuly Ruzimuhammad
Group	SE-2501

Publication: Updated deployment verified 2026-10-27.

Deployed website: https://ramazanutegen.github.io/assignment1web/

Verified website source: 319cad969277; GitHub Pages build and deployment completed successfully.

Prepared for all three teammates.

All three members use the same complete project and report. The split below is a defence preparation plan, not an invented record of individual authorship. Every member should understand the entire application.

Ramazan Utegen | Home and Aktau

Demonstrate combined search/category filters, saved cities, the empty state, DOM selection and updates. Explain switch, filter, map, forEach and localStorage. Practise adding a search tag and changing a category.

Ziyadinkhan Kudaibergenuly | Astana and About Us

Demonstrate keyboard navigation, day/night themes and per-page ratings. Explain keydown, preventDefault, focus wrapping, aria-pressed, CSS variables and Bootstrap theme inheritance. Practise a keyboard action or rating message.

Ruzimuhammad | Almaty and Shymkent

Demonstrate the three-step planner and user-enabled chime. Explain changeStep and its callback, destination objects and getRoute, the explicit route loop, async resume, sound cancellation and audio cleanup. Practise a route-stop change.

Shared rehearsal. Run an invalid form, correct it, navigate Back/Next without losing data, build a two-day route, reset it, rate a guide, navigate the gallery and close the contact popup with Escape. Trace each action from listener to data to DOM to CSS.

Individual submission. Each member uploads KZ-Travel-Assignment-5-Report.pdf and KZ-Travel-Assignment-5.zip in the LMS and attends the practice-lesson defence. See docs/assignment-5-teammate-guide.md for detailed questions, expected answers and live-coding practice.

Select, update and rate.

Part 1.1: DOM selection, dynamic text, class-based styling and the chosen rating-star option for attribute manipulation.

1. Select the rating group and its five stars with querySelector/querySelectorAll.

2. A click updates the selected value, CSS class, aria-pressed and textContent feedback.

3. Store the page-specific rating locally; reload to restore it. User text is never inserted as HTML.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

DOM selection and star-rating updates

js/experience.js

```
230     document.querySelectorAll('[data-rating]').forEach(rating => {
231         const stars = rating.querySelectorAll('[data-star]');
232         const message = rating.querySelector('[data-rating-message]');
233         const key = 'rating-' + rating.dataset.rating;
234         let selected = Number(storage.read(key, 0));
235         if (!Number.isInteger(selected) || selected < 0 || selected > 5) selected =
    0;
236         function renderRating() {
237             stars.forEach(star => {
238                 star.classList.toggle('is-selected', Number(star.dataset.star) <=
    selected);
239                 star.setAttribute('aria-pressed', String(Number(star.dataset.star)
    === selected));
240             });
241             message.textContent = selected ? `Your rating: ${selected}/5. Thanks for
    exploring with us!` : 'Choose your rating';
242         }
243         stars.forEach(star => star.addEventListener('click', () => {
244             selected = Number(star.dataset.star);
245             storage.write(key, selected);
246             renderRating();
247             animate(message);
248             playChime();
249         }));
250         renderRating();
251     });
252
```

Source-code screenshot, rendered from the actual project file with original line numbers.

KZ TRAVEL / ASSIGNMENT 5 / 02

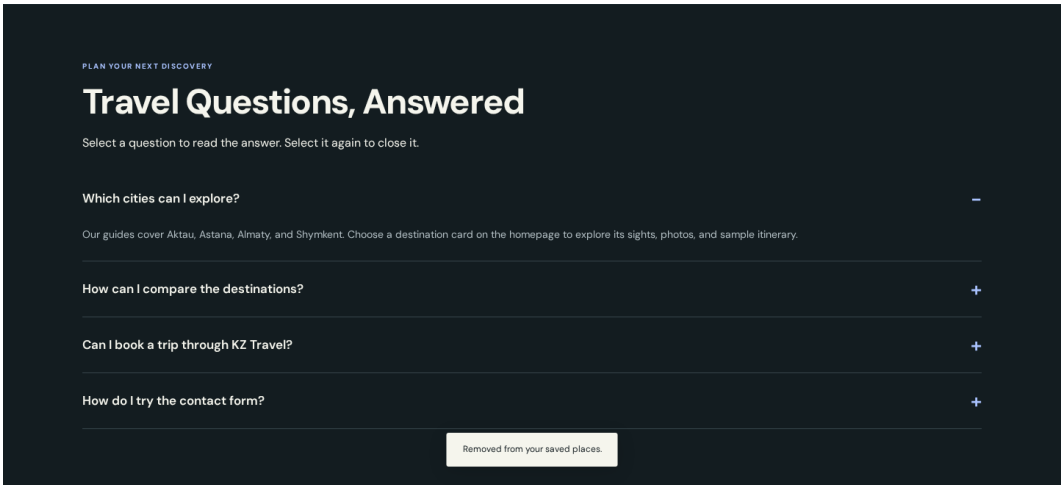
A guide for day and night.

Part 1.1: dynamic styling; chosen option: functional Day/Night switch.

1. The moon/sun button calls applyTheme and changes the body data-theme attribute.

2. CSS variables recolour surfaces, text, borders and controls across all six pages.

3. Persist the theme locally. Keep the earlier Sand/Sky/Steppe background cycle working with matching night colours.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Persistent day and night themes

js/experience.js

```
118     function applyTheme(theme) {
119         document.body.dataset.theme = theme;
120         document.body.dataset.bsTheme = theme === 'night' ? 'dark' : 'light';
121         themeToggle?.setAttribute('aria-pressed', String(theme === 'night'));
122         themeToggle?.setAttribute('aria-label', theme === 'night' ? 'Switch to day
    theme' : 'Switch to night theme');
123         if (themeToggle) themeToggle.textContent = theme === 'night' ? '☀' : '☾';
124         document.dispatchEvent(new Event('kz-theme-change'));
125     }
126     applyTheme(storage.read('theme', 'day') === 'night' ? 'night' : 'day');
127     themeToggle?.addEventListener('click', () => {
128         const theme = document.body.dataset.theme === 'night' ? 'day' : 'night';
129         applyTheme(theme);
130         storage.write('theme', theme);
131     });
132
```

Source-code screenshot, rendered from the actual project file with original line numbers.

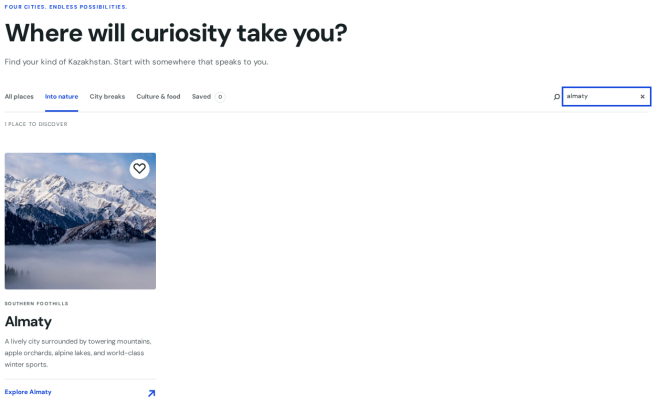
Make discovery respond.

Part 1.2: addEventListener, keyboard handling and switch statements.

1. Bind click/input listeners to category buttons, saved hearts and destination search.

2. Use a switch to choose category logic; combine it with the search query and update results without reload.

3. Use keydown and a switch for four arrow keys, Home and End. Move focus through navigation links and preserve normal Tab behaviour.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Keyboard listeners and switch logic

js/experience.js

```
134     const menuLinks = Array.from(document.querySelectorAll('#main-navigation .nav-
    link'));
135     for (let index = 0; index < menuLinks.length; index += 1) {
136         menuLinks[index].addEventListener('keydown', event => {
137             let nextIndex;
138             switch (event.key) {
139                 case 'ArrowRight': case 'ArrowDown': nextIndex = (index + 1) %
    menuLinks.length; break;
140                 case 'ArrowLeft': case 'ArrowUp': nextIndex = (index - 1 +
    menuLinks.length) % menuLinks.length; break;
141                 case 'Home': nextIndex = 0; break;
142                 case 'End': nextIndex = menuLinks.length - 1; break;
143                 default: return;
144             }
145             event.preventDefault();
146             menuLinks[nextIndex].focus();
147         });
148     }
149
```

Source-code screenshot, rendered from the actual project file with original line numbers.

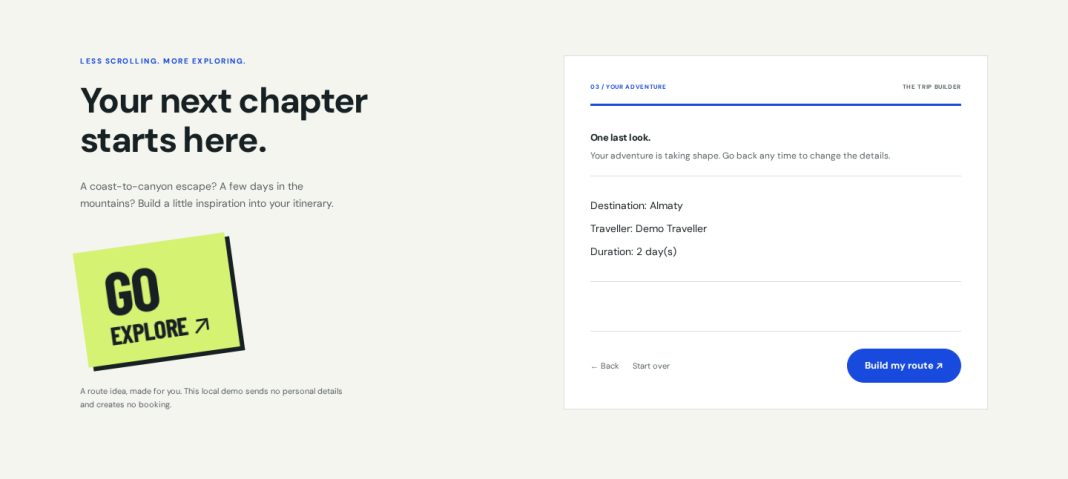
One journey, three steps.

Part 1.2: chosen callback option: multi-step form with Next and Back.

1. Create destination, traveller-details and review fieldsets. Hidden steps remain in the DOM.

2. changeStep validates the current step, changes its index, then invokes the onStepChanged callback. Invalid controls receive accessible feedback and focus.

3. Pass showStep as that callback. Next and Enter share the submit handler; Back preserves inputs. Inactive fieldsets are disabled. Start over clears input and generated output.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Multi-step form: validation and rendering callback

js/experience.js

```
332         function changeStep(direction, onStepChanged) {
333             const issue = direction > 0 ? validationIssue(step) : null;
334             if (issue) {
335                 showError(issue);
336                 return;
337             }
338             step = Math.max(0, Math.min(2, step + direction));
339             onStepChanged(step);
340             focusStep();
341         }
342         function buildRoute() {
343             // Recheck every required value before producing the final result.
344             for (let index = 0; index < 2; index += 1) {
345                 const issue = validationIssue(index);
346                 if (issue) {
```

Source-code screenshot, rendered from the actual project file with original line numbers.

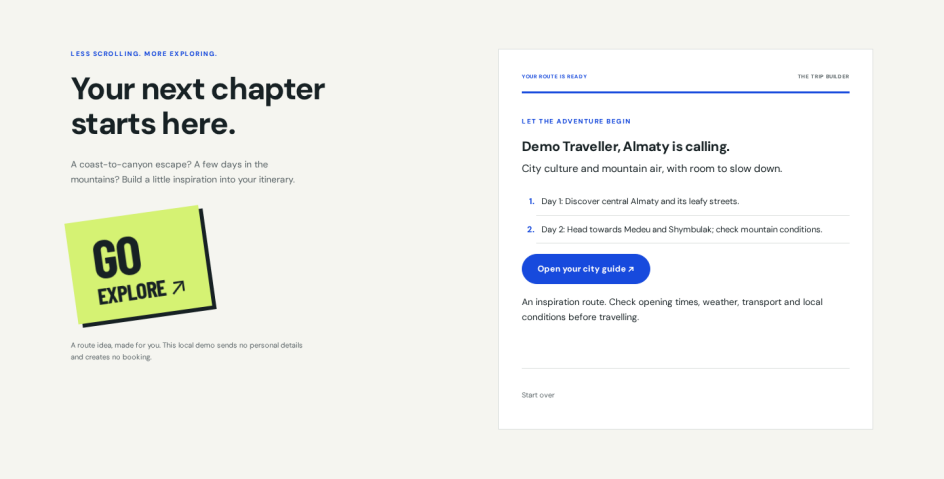
Turn data into a route.

Part 1.3: objects and methods, arrays, loops and higher-order functions.

1. Represent each city as an object containing its name, category, description and stops array.

2. Add getRoute(days), which returns this.stops.slice(0, days). Use map/filter/forEach to process collections.

3. Build the selected route with an explicit for loop, createElement and textContent. A two-day choice generates exactly two day items.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Objects, methods, higher-order functions and a loop

js/experience.js

```
18      const destinations = [
19          { id: 'aktau', name: 'Aktau', category: 'nature', tags: 'sea coast desert
    canyons mangystau',
20              description: 'A Caspian escape, with wide horizons and dramatic
    landscapes.',
21              stops: ['Walk the Caspian waterfront and discover Aktau.', 'Explore
    Mangystau chalk formations with a local guide.', 'Make time for the Bozjyra
    landscapes and the open steppe.'] },
22          { id: 'astana', name: 'Astana', category: 'city', tags: 'capital
    architecture city river',
23              description: 'A fresh perspective on architecture, art and life on the
    steppe.',
24              stops: ['Explore the Baiterek area and Nurzhol Boulevard.', 'Discover
    Khan Shatyr and the city architecture.', 'Take a slower walk beside the Ishim
    River.'] },
```

js/experience.js

```
32          ...destination,
33          getRoute(days) { return this.stops.slice(0, days); }
34      }));
```

js/experience.js

```
357             const stops = planner.querySelector('[data-trip-stops]');
358             stops.replaceChildren();
359             // Explicit loop demonstrates array iteration while generating actual
    DOM nodes.
360             const route = place.getRoute(days);
361             for (let index = 0; index < route.length; index += 1) addText(stops,
    'li', `Day ${index + 1}: ${route[index]}`);
```

Source-code screenshot, rendered from the actual project file with original line numbers.

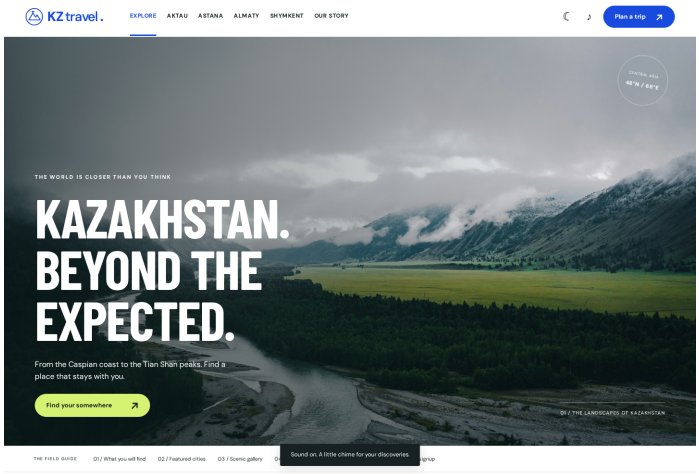
A small sound of discovery.

Part 1.3: JavaScript-triggered sound effects, controlled by the user.

1. Sound starts disabled. Clicking the music note enables audio and previews the chime.

2. A sine oscillator changes from 660 to 880 Hz. A short gain envelope fades the note out.

3. Saving, rating or completing a route can trigger it. A new sound replaces the previous oscillator. Muting immediately disconnects it and cancels pending resume requests; onended cleans up nodes.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

A short, user-triggered Web Audio chime

js/experience.js

```
 82            const oscillator = audioContext.createOscillator();
 83            const gain = audioContext.createGain();
 84            const now = audioContext.currentTime;
 85            oscillator.type = 'sine';
 86            oscillator.frequency.setValueAtTime(660, now);
 87            oscillator.frequency.setValueAtTime(880, now + 0.09);
 88            gain.gain.setValueAtTime(0.0001, now);
 89            gain.gain.exponentialRampToValueAtTime(0.12, now + 0.015);
 90            gain.gain.exponentialRampToValueAtTime(0.0001, now + 0.23);
 91            oscillator.connect(gain);
 92            gain.connect(audioContext.destination);
 93            activeSound = oscillator;
 94            oscillator.onended = () => {
 95                oscillator.disconnect();
 96                gain.disconnect();
 97                if (activeSound === oscillator) activeSound = null;
 98            };
 99            oscillator.start(now);
100            oscillator.stop(now + 0.24);
101        } catch {
102            if (soundEnabled && request === soundRequest) notify('Your browser could
    not play the sound.');
103        }
```

Source-code screenshot, rendered from the actual project file with original line numbers.

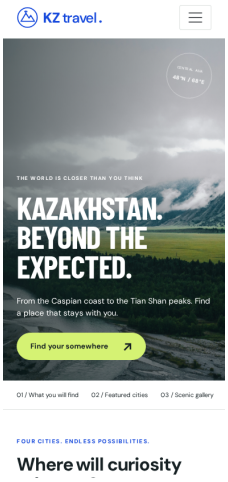
Motion with a purpose.

Parts 1.3-1.5: event-triggered animation, separation of concerns, clean structure and theme integration.

1. Restart ui-enter after filtering, rating or changing a step; use a pulse for a saved heart.

2. CSS controls transitions, keyframes and progress animation; prefers-reduced-motion disables movement.

3. Keep markup in HTML, presentation in editorial.css, and behaviour in experience.js. Use meaningful helper functions, local assets, responsive layouts and travel-specific content.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Event-triggered movement with reduced-motion support

js/experience.js

```
36      function animate(element) {
37          if (!element) return;
38          element.classList.remove('ui-enter');
39          // Restart the event-triggered CSS animation after the DOM update.
40          void element.offsetWidth;
41          element.classList.add('ui-enter');
42      }
43
```

css/editorial.css

```
1795 @media(prefers-reduced-motion:reduce) {
1796      html {
1797          scroll-behavior: auto
1798      }
1799
1800      .ui-enter,
1801      .save-place.is-pulsing {
1802          animation: none
1803      }
1804
1805      .editorial *,
```

Source-code screenshot, rendered from the actual project file with original line numbers.

Keep every view of Kazakhstan.

Additional DOM enhancement: clickable gallery thumbnails while retaining the original manual galleries.

1. Retain the five nine-image Bootstrap carousels, original image sources, arrows and indicators.

2. Add a button for every thumbnail, targeting the matching slide through Bootstrap data attributes.

3. On slid.bs.carousel, update the thumbnail class and aria-pressed. Manual, thumbnail and keyboard navigation remain in sync.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Existing carousel and new thumbnail state

js/experience.js

```
220     document.querySelectorAll('[data-gallery-for]').forEach(thumbnails => {
221         const carousel = document.getElementById(thumbnails.dataset.galleryFor);
222         carousel?.addEventListener('slid.bs.carousel', event => {
223             thumbnails.querySelectorAll('button').forEach((button, index) => {
224                 button.classList.toggle('is-active', index === event.to);
225                 button.setAttribute('aria-pressed', String(index === event.to));
226             });
227         });
228     });
229
230     document.querySelectorAll('[data-rating]').forEach(rating => {
```

Source-code screenshot, rendered from the actual project file with original line numbers.

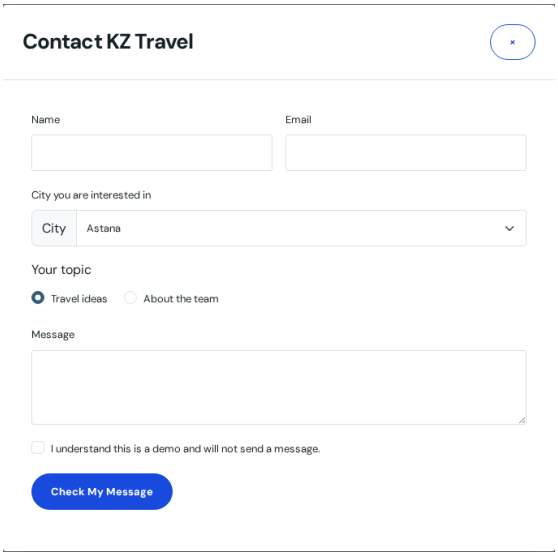
Earlier work, still working.

The redesign preserves the existing theme, all original image files and references, city content, sample routes, tables and team credits. The shared Bootstrap grid and the earlier CSS-only media-query card exercise remain.

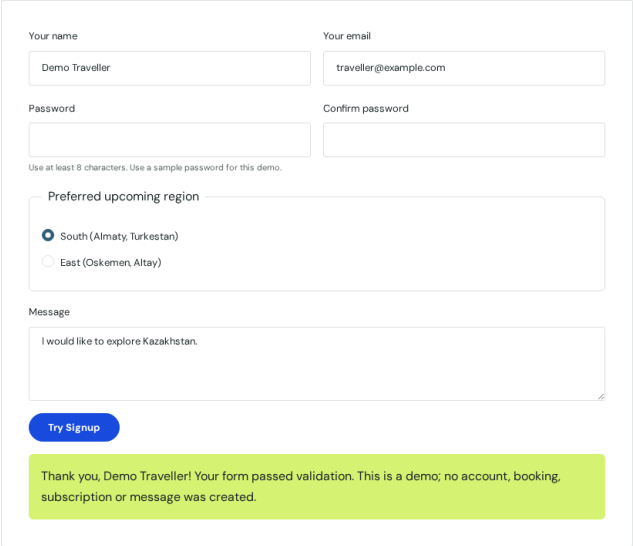
1. Legacy forms still validate required fields, email, password length and confirmation; successful validation stays on the page.

2. The homepage FAQ opens/closes; the About Us dialog closes with Escape and returns focus to Contact Us.

3. The clock updates in Asia/Almaty (UTC+5). Background cycling now appears in the footer.



Retained contact dialog; the screenshot shows the original form fields in the new design.



A valid sample form displays confirmation without transmitting data or creating a subscription.

Verification, reflection and publication.

Review environment: automated Playwright tests in Chromium and Firefox on Linux, at 1440 x 1000 desktop and 390 x 844 phone viewports. All six pages are exercised in each combination. The browser timezone is America/New_York to verify the Kazakhstan clock independently.

Published-site recheck: published HTML and local assets returned HTTP 200 and matched the current project. Deployment and browser evidence is saved with this submission.

Check	Observed result
Six desktop pages	No horizontal overflow or broken loaded images
24 page/browser layouts	Six pages fit desktop and phone sizes in both browsers
Search, filters and saved places	Combined criteria, empty state and persistence work
Planner and callbacks	Accessible errors, Enter, Back, safe text, 1/2/3-day routes, reset
Theme, rating and navigation	Persistence, star feedback and arrow-key focus work
Legacy functions	Form validation, FAQ, galleries, popup and focus return
Static source check	Resources, anchors, IDs, credits and all original photos
Audio and fallback checks	No autoplay/overlap, pending mute, storage disabled, reduced motion

Challenges and solutions. Syncing the checkout first preserved newer team work. Hidden fieldsets keep input values; disabled inactive steps and explicit focus make the planner usable with a keyboard. textContent safely displays names. Custom and Bootstrap themes now stay in sync, including error colours. A request counter cancels audio that resumes after muting. Portable fonts let this report rebuild on Linux, Windows or macOS.

Reflection and defence. Each member should trace an event from its listener to its callback, data processing, DOM update and CSS effect. Practise explaining this, getRoute, switch, filter/map/forEach, the explicit for loop, aria-pressed, hidden, localStorage and reduced-motion handling. Rehearsal notes are included in English and Russian.

Part 2 - updated website deployment. The redesign was committed and pushed to main. GitHub Pages deployed source 319cad969277. The published site was verified on 2026-10-27: all six pages and their local assets were checked, and live browser tests covered discovery, saved places, themes, ratings, keyboard navigation, the planner, galleries, FAQ and the contact dialog.

https://ramazanutegen.github.io/assignment1web/

Deployment evidence: successful GitHub Pages workflow; detailed live checks and asset hashes are in docs/assignment-5/deployment.json.

Submission package: the site is published; the PDF and ZIP are updated. Every team member submits both files and attends the practice-lesson defence. Check the deadline and its time zone directly in the LMS; this report does not verify the deadline.